GISMA UNIVERSITY OF APPLIED SCIENCES

B103 Databases & Big Data

University Management System

Arjun Bijoy

Student ID: GH1044696

GitHub Repository: https://github.com/arjunbijoy1917r-cmyk/university_database_project.git

Video Demonstration: https://youtu.be/cdi2sXNzxhg

Submission Deadline: April 9, 2026

1. Introduction

This report presents the design and implementation of a University Management System relational database, developed as part of the B103 Databases & Big Data module. This project shows the database concepts like schema design, normalisation, constraint enforcement, and SQL operations.

The University Management System was selected because of the relationship between the data of real-world. Universities deals the information from students, instructors, courses, modules, and enrolments. This require careful relational modelling and well-structured data handling.

1.1 Objectives

- Design a relational database schema for a university context.
- Implement the schema in MySQL with constraints, primary keys, and foreign keys.
- Providing the sample data for the database.
- Demonstrate SQL operations like CRUD, JOIN, and aggregation queries.

2. System Design

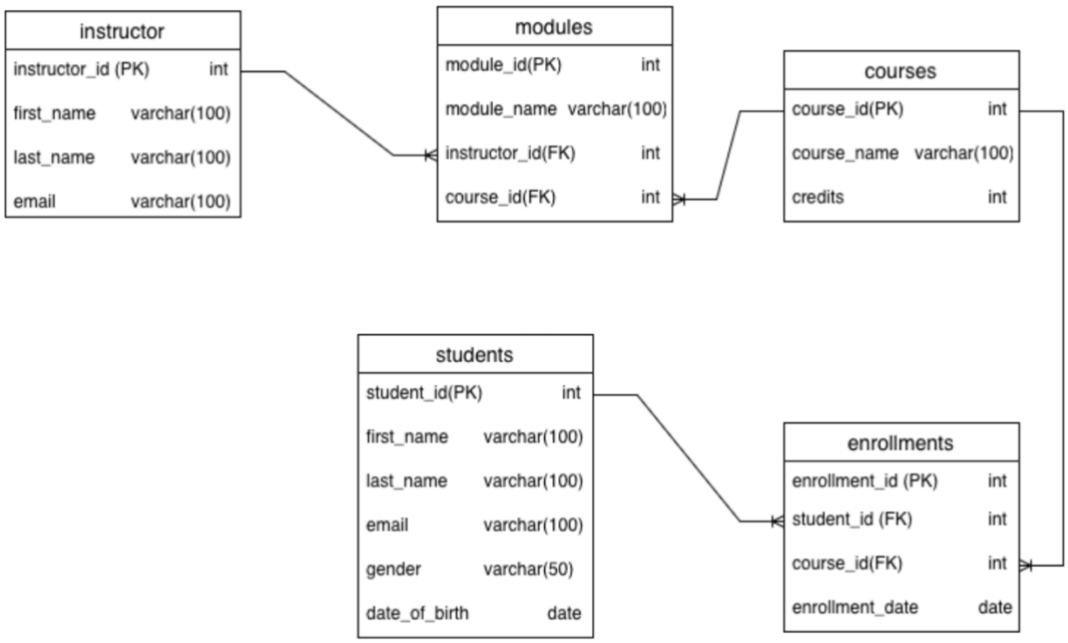
Considering a university context, the design process involved identifying the core entities, defining their attributes, and establishing the relationships between them.

2.1 Entities and Attributes

Five entities were used in the University Management database:

- Students —individuals enrolled at the university (student_id, first_name, last_name, email, gender, date_of_birth).
- Instructors — lecturers who teaches modules (instructor_id, first_name, last_name, email).
- Courses — programmes of study (course_id, course_name, credits).
- Modules —individual subjects within a course, assigned to an instructor (module_id, module_name, instructor_id, course_id).
- Enrollments —data of students and the courses they are enrolled in (enrollment_id, student_id, course_id, enrollment_date).

2.2 ER Diagram



The Entity-Relationship (ER) diagram has the following relationships:

- A Course has many Modules .Each module belongs to exactly one course. (one-to-many)
- An Instructor teaches many Modules .Each module is assigned to one instructor. (one-to-many)
- A Student can enrol in many Courses, and a Course can have many Students .In this case enrollments junction table is used. (many-to-many)
- To avoid redundancy the Enrollments table resolves the many-to-many relationship between Students and Courses by storing the enrollment_date as an additional attribute.

Entity	Primary Key	Foreign Keys	Relationship
students	student_id	—--	Enrolled in courses
instructors	instructor_id	—--	Teaches modules
courses	course_id	—--	Contains modules
modules	module_id	instructor_id, course_id	Belongs to one course and one instructor
enrollments	enrollment_id	student_id, course_id	Links students to courses

2.3 Normalisation

- 1NF: All attributes are single. There are no groups that repeat and there is a clear primary key for each table.
- 2NF: All non-key attributes depends completely on the primary key and there are no partial dependencies.
- 3NF: There are no transitive dependencies.

The many-to-many relationship between students and courses was sorted by the enrolments table, which stopped problems with updates and new entries.

3. Implementation

The database was implemented in DBeaver with MySQL. Each steps of implementation is explained below.

3.1 Database and Table Creation

The database university_db was created using CREATE DATABASE and selected with the USE command:

```
CREATE DATABASE university_db;
USE university_db;
```

Then the five tables are created. The students table stores student information with an auto-incrementing primary key. To prevent duplicate registrations the email column has a UNIQUE constraint, and NOT NULL constraints ensure data integrity on essential fields.

```
CREATE TABLE students(
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(100)NOT NULL,
    last_name VARCHAR(100 ) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    Gender VARCHAR(50),
    date_of_birth DATE
)
```

The instructors table follows the same pattern. The courses table have a credits column representing the scores of each course. The modules table links both courses and instructors. The enrollments table records the relationship between students and courses and capture the enrollment date.

```
CREATE TABLE enrollments(
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT
    student_id INT,
    course_id INT,
    enrollment_date DATE
)
```

3.2 Foreign Key Constraints

Foreign keys are added using ALTER TABLE statements after all tables were created to avoid forward reference issues during table creation.

```sql
ALTER TABLE modules
ADD CONSTRAINT fk_modules_course
FOREIGN KEY (course_id ) REFERENCES courses(course_id);
ALTER TABLE modules
ADD CONSTRAINT fk_modules_instructor
FOREIGN KEY (instructor_id) REFERENCES instructors(instructor_id);

ALTER TABLE enrollments
ADD CONSTRAINT fk_enrollments_student
FOREIGN KEY (student_id) REFERENCES students(student_id);
ALTER TABLE enrollments
ADD CONSTRAINT fk_enrollments_course
FOREIGN KEY (course_id) REFERENCES courses(course_id);
```

3.3 Sample Data Insertion

Sample data are inserted to provide a real world scenario. It includes four instructors, five courses, four students, four modules, and six enrolment records were added:

```sql
INSERT INTO instructors (first_name, last_name, email) VALUES
('Arjun', 'Smith', 'arjun.452@gmail.com'),
('Carlsen', 'Brown', 'emily.brown@yahoo.com'),
('Jackson', 'Clark', 'michael.123@uni.com'),
('Fredrick','Dannis', 'fdrick@gmail.com');

INSERT INTO courses (course_name, credits) VALUES
('Business Management', 8),
('Foreign Language', 3),
('Database Systems', 4),
('Data Science', 3),
('Software Engineering', 5);
```

4. Results and Query Demonstration

A set of SQL queries was developed to validate the database design and to show its functionality. The queries cover standard CRUD operations.

4.1 JOIN Query

This query gets a list of student names, course names, and enrolment dates by joining the enrollments, students, and courses tables:

```sql
SELECT
    s.first_name,
    s.last_name,
    c.course_name,
    e.enrollment_date
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id;
```

Output :- a table joining student records with their corresponding course and enrolment date:

	first_name	last_name	course_name	enrollment_date
1	Arjun	Bijoy	Business Management	2026-01-10
2	Pinky	Gandhi	Foreign Language	2026-01-12
3	Muhammed	Singh	Database Systems	2026-01-15
4	David	Verma	Data Science	2026-01-18
5	Arjun	Bijoy	Foreign Language	2026-01-20
6	Pinky	Gandhi	Database Systems	2026-01-22

4.2 Aggregation Query

This query uses GROUP BY and COUNT to calculate how many students are enrolled in each course.

```sql
SELECT
    c.course_name,
    COUNT(e.student_id) AS total_students
FROM enrollments e
JOIN courses c ON e.course_id = c.course_id
GROUP BY c.course_name;
```

	course_name	total_students
1	Business Management	1
2	Data Science	1
3	Database Systems	2
4	Foreign Language	2

4.3 ORDER Query

This query sorts all courses by their credit value, from highest to lowest.

```
SELECT course_name, credits
FROM courses
ORDER BY credits DESC;
```

	course_name	credits
1	Business Management	8
2	Software Engineering	5
3	Database Systems	4
4	Foreign Language	3
5	Data Science	3

4.4 DELETE Operation

This query demonstrates a targeted DELETE operation, which removes a particular enrollment record using its primary key. This simulates a student dropping from a course.

```
DELETE FROM enrollments
WHERE enrollment_id = 6;
```

After this deletion, we used a query SELECT*FROM students to check all the data, in the students table to ensure data integrity i.e. the students table remained unaffected by the deletion in the enrollments table.

	dent_id	first_name	last_name	email	Gender	date_of_birth
1	1	Arjun	Bijoy	arj234@gmail.com	male	2006-05-1
2	2	Pinky	Gandhi	rahul@gmail.com	female	2005-08-1
3	3	Muhammed	Singh	priya@yahoo.com	male	2003-02-2
4	4	David	Verma	vd828@gmail.com	male	2005-11-2

5. Challenges and Solutions

5.1 Foreign Key Ordering

Foreign key constraints were inside the CREATE TABLE command at first. This resulted in an error because MySQL demands the existence of the referenced table before creating the constraint on the referring table.

Solution: The foreign key constraints were removed from the CREATE TABLE command and added ALTER TABLE command after the creation of all five tables.

5.2 Many-to-Many Relationship Design

The relationship between students and courses is actually many-to-many. A student can enrol in multiple courses, and a course can have many students. Representing this directly like holding a list of course IDs within the student's record would be very hard while rendering queries.

Solution: A junction table (enrolment) was added so that the many-to-many relationship could be separated into two one-to-many relationships. This approach helped to solve this problem in relation database design. Thus able store those attributes which are specifically relate to the relationship.

5.3 Data Integrity and Constraint Enforcement

Challenge: Without constraints, it would be possible to insert logically inconsistent data such as an enrollment referencing a student or course that does not exist, or two students sharing the same email address.

Solution: UNIQUE constraints were applied to email fields in both the students and instructors tables. NOT NULL constraints were applied to all mandatory fields. Foreign key constraints ensure referential integrity across all inter-table relationships, with the database engine rejecting any invalid references at insertion time.

6. Conclusion and Future Work

6.1 Conclusion

In conclusion, this project has been successful in creating an efficient relational database that can be used for the University Management System. This design followed the guidelines of normalization for suitable primary keys, UNIQUE, foreign keys and NOT NULL constraints.

The SQL statements were implemented to carry out the operations that include insertions, SELECT statements using multiple table JOIN, GROUP BY and aggregate functions, ORDER BY, and deletion operations.

6.2 Future Work

Some extensions would help in improving the future versions of system:

- A table for Departments can be added to classify professors under different academic departments .A departments table can group instructors and a further layer for organising structure.
- A grades table can be added to keep track of grades of individual modules to calculate GPAs.
- Indexing: Creating indexes on certain fields which are most commonly searched, like student's email or the course_name will speed up the execution time.